

Implement a production-ready TikTok LIVE gift wheel called **Spin the Screen** in the existing livestream project.

Do not return only a plan. Inspect the repository, summarize the relevant architecture and then complete the implementation, testing and documentation. Preserve unrelated existing behavior and user changes.

## Primary objective

Create a transparent `1080×1920` wheel overlay in which:

- A qualifying completed Gift or Gift streak produces one spin.
- The wheel determines the cosmetic effect.
- Gift value determines the effect's size, duration and persistent recognition.
- Large Gifts always receive large visible impact regardless of which result is selected.
- The overlay uses the existing gesture controller for face tracking.
- Aply carries only discrete events—not face coordinates or animation frames.

The viewer-facing explanation is:

**The wheel picks the effect. Bigger Gifts make it spread farther and leave your name longer.**

## Non-negotiable production pipeline

TikTok LIVE Studio remains the master compositor.

The current route is:

1. Physical camera, gameplay and overlays are separate sources inside TikTok LIVE Studio.
2. TikTok LIVE Studio applies native face enhancement and Interactive Gift effects.
3. LIVE Studio opens a flattened projector of the completed TikTok canvas.
4. OBS captures that projector window as its full-screen source.

Do not reverse this pipeline.

Specifically:

- Do not move the primary composition into OBS.
- Do not replace the complete TikTok canvas with an OBS Virtual Camera.
- Do not change the physical-camera routing simply to implement this wheel.
- Do not interfere with TikTok's ability to recognize the camera and place its native Interactive Gifts on the creator's face.
- The new wheel must be a transparent overlay loaded inside TikTok LIVE Studio using the project's existing overlay method.
- The completed effect will naturally appear in the LIVE Studio projector and therefore in OBS.

# Repository audit

Before changing code, locate and document:

- Gift/TikFinity event ingestion
- Gift streak/combo handling
- Existing Ably channels and message contracts
- Existing backend/controller process
- Existing overlay renderer
- Existing operator/control interface
- Existing persistence mechanism
- Existing gesture controller
- How the gesture controller acquires and distributes video frames
- Whether it already produces face, pose or hand landmarks
- Its local IPC/state/event mechanism
- Current `1080×1920` geometry and camera placement
- How the overlay is loaded into TikTok LIVE Studio
- Existing audio-effect routing

Reuse those components rather than introducing parallel infrastructure.

The existing gesture controller is part of the required solution. Do not create an independent face-tracking application.

If the gesture-controller source is not actually present in the repository or accessible workspace, that is a genuine blocker: identify the exact missing component/path before altering camera acquisition.

## Required implementation order

Work in these checkpoints, keeping the project runnable after each one:

1. Core event normalization, tier resolver and deduplication
2. Authoritative wheel-result selection and persistence
3. Wheel state machine and separate rendering lanes
4. Complete vertical slice using Stamp Your Name
5. Integrate face tracking into the existing gesture controller
6. Connect live Gift events and queue handling
7. Implement one polished default for each remaining result
8. Add operator controls and calibration
9. Harden reconnect, refresh and failure recovery
10. Run automated tests, synthetic burst tests and production build

Do not build dozens of effect variants before the core vertical slice is reliable.

# Gift tiers

Final coin thresholds must be configurable outside the code. Do not scatter threshold constants throughout the implementation.

Use these four internal tiers:

## Spin

- Local camera-region effect
- Small name treatment
- Short persistence

## Power Spin

- Whole camera or one major canvas region
- More prominent name treatment
- Medium persistence

## Takeover Spin

- Brief full-canvas impact
- Premium persistent signature
- Subtle ambient state after the impact

## Legend Spin

- Premium full-canvas entrance
- Session-level recognition
- Gold or otherwise unmistakable Legend treatment

Communicate tiers using text, iconography, scale, shape and motion—not color alone.

Do not require viewers to understand coin conversions. During an event, display a concise treatment such as:

@USERNAME · POWER SPIN  
STAMP

# Gift normalization

Generate one wheel event from a completed Gift action.

Requirements:

- Streakable Gifts trigger when the completed streak/end event is known.
- Use total Gift value multiplied by the final repeat count.
- Non-streakable Gifts trigger once normally.
- Intermediate repeat updates must not create separate spins.
- Use a stable event ID.
- Deduplicate repeated deliveries.
- Only configured qualifying Gifts trigger the wheel.
- Test/manual events must be clearly marked as simulation events.

Create an adapter between the existing Gift provider and a normalized internal event. Do not bind the wheel directly to provider-specific field names throughout the application.

## Authoritative result selection

The backend/controller—not each overlay client—must resolve:

- Tier
- Enabled-result pool
- Selected result
- Effect variant
- Style/random seed
- Signature class
- Signature creation and expiry times
- Session ID
- State revision

Persist the resolution before publishing it.

Use secure local randomness such as `crypto.getRandomValues()` when selecting the result. Once selected, reconnects and multiple clients must receive the same outcome.

All currently enabled wheel slices must have equal visible size and equal probability.

If a result is disabled:

- Remove it from the visible wheel.
- Remove it from the selection pool.
- Redistribute probability equally among the remaining results.

If only one result is enabled, skip the fake spin and trigger that result directly.

## Minimal Ably usage

Reuse the existing Ably setup, authentication and channel conventions.

Publish one authoritative resolved-spin command per qualifying Gift. A suitable schema is:

```
{
  "type": "wheel.spin",
  "schemaVersion": 1,
  "revision": 1042,
  "eventId": "stable-event-id",
  "sessionId": "session-id",
  "issuedAt": 1784760000000,
  "simulation": false,
  "user": {
    "id": "platform-user-id",
    "username": "@ViewerName",
    "avatarUrl": "https://..."
  },
  "gift": {
    "id": "gift-id",
    "name": "gift-name",
    "repeatCount": 1,
    "totalValue": 1000
  },
  "tier": "takeover",
  "result": "stamp",
  "variant": "rubber-ink",
  "styleSeed": "resolved-seed",
  "signature": {
    "class": "premium",
    "createdAt": 1784760000000,
    "expiresAt": 1784761200000
  }
}
```

Ably may also carry low-frequency operator commands and state synchronization when genuinely required.

Do not publish through Ably:

- Face landmarks
- Face-anchor coordinates
- Gesture-controller frame data
- Particle positions
- Wheel rotation frames
- Animation progress
- Per-frame transforms
- Routine animation-started/completed messages
- High-frequency overlay heartbeats

All continuous visual and tracking state remains local.

Do not expose an Ably API key or other secret in browser code. Preserve or improve existing token authentication.

## Rendering architecture

Create separate rendering lanes:

1. **Receipt lane**

Immediately acknowledges that an event was accepted or queued.

2. **Wheel lane**

Runs one wheel at a time.

3. **Impact lane**

Runs the short result reveal and primary impact.

4. **Ambient lane**

Runs longer low-opacity effects without blocking the next spin.

5. **Signature lane**

Manages persistent names independently.

Only the wheel and short impact lock the foreground sequence. Ambient effects and persistent signatures must not block the next Gift.

Recommended timings:

- Receipt acknowledgment: within 300–500 ms
- Wheel entrance and spin: 1.8–2.2 seconds
- Result reveal: 400–600 ms
- Standard impact: 2–4 seconds
- Legend impact: maximum 6 seconds
- Strong opaque full-screen obstruction: preferably under 4 seconds
- Ambient effect: 10–45 seconds at readable opacity
- Persistent recognition: controlled separately by tier

Create a deterministic state machine such as:

```
queued → acknowledged → spinning → impact → settled → complete
```

Each stage must have:

- Explicit cleanup

- Cancellation support
- A watchdog timeout
- Failure recovery that releases the queue
- No orphaned timers or DOM nodes

## Queue handling

The active wheel is never interrupted.

When an event arrives:

1. Acknowledge it immediately.
2. Show its tier and queue position if waiting.
3. Persist it in the authoritative queue.
4. Process it in order unless the existing project has a better documented priority rule.

Example:

```
@NAME LOCKED A POWER SPIN · #3
```

Requirements:

- No silent event loss.
- Completed streak duplicates collapse into one event.
- Queue remains recoverable after overlay refresh.
- Ambient effects do not block queue progress.
- Takeover/Legend impacts may temporarily suppress lower-tier ambient effects without deleting persistent names.
- Implement a configurable rapid-spin presentation when backlog exceeds a threshold.
- Document and expose any aggregation/overflow policy.
- Never silently discard an overflow event.

Operator controls must include:

- Pause intake
- Resume intake
- Skip current
- Drain queue
- Clear queue with confirmation
- Emergency clear transient effects
- Clear persistent signatures separately

Emergency clearing transient effects must not erase promised persistent recognition.

# Effect plugin contract

Implement effects through a shared registry/interface rather than one giant conditional.

Each effect should support concepts equivalent to:

```
interface WheelEffect {  
  id: WheelResultId;  
  label: string;  
  prepare(context: EffectContext): Promise<void> | void;  
  playImpact(context: EffectContext, signal: AbortSignal): Promise<void>;  
  startAmbient(context: EffectContext): AmbientHandle | null;  
  createSignature(context: EffectContext): SignatureSpec;  
  cleanup(reason: CleanupReason): void;  
}
```

Every effect receives the same resolved event, tier envelope, geometry, style seed, asset loader, tracking provider and cleanup signal.

Implement one polished default variant for each result. Build extension points for future variants, but do not turn every example into required launch scope.

## Wheel results

Use these eight results:

1. CAMERA CURSE
2. STAMP YOUR NAME
3. COLOR FLOOD
4. WEATHER CONTROL
5. BREAK THE GLASS
6. STEAL THE SPOTLIGHT
7. CLAIM THE FRAME
8. AVATAR INVASION

Use short labels on the wheel:

- Curse
- Stamp
- Flood
- Weather
- Crack
- Spotlight
- Frame



- Avatar

## Camera Curse

Launch version:

- Overlay treatment confined to the calibrated camera rectangle
- Bars, warning frame, ink or another coherent “camera cursed” treatment
- CAMERA CURSED BY @username
- Tier controls size, intensity and persistence

Do not falsely claim that a transparent overlay can filter TikTok’s camera pixels. True camera manipulation should only use capabilities already provided safely by the gesture/camera controller.

## Stamp Your Name

This is the flagship result and must be especially polished.

### Base sequence

1. Display @username IS LEAVING THEIR MARK .
2. Bring an oversized rubber stamp into view.
3. Target the tracked forehead anchor when a stable face is available.
4. Compress the stamp against the virtual camera glass/face position.
5. Play a restrained impact and canvas bump.
6. Lift the stamp.
7. Reveal a readable rubber-ink username impression.
8. Let the face-anchored impression follow naturally for its configured active period.
9. Detach it from the face.
10. Animate it into the persistent signature system.

### Tier behavior

#### Spin

- Small tracked stamp near the forehead or upper camera region
- Short face-following duration
- Small persistent signature

#### Power Spin

- More prominent tracked stamp
- Larger camera-region impact
- Medium persistent signature

#### Takeover Spin

- Brief signature across a gameplay-safe portion of the canvas

- A tracked face seal during the ambient stage
- Premium persistent signature

### **Legend Spin**

- Full-canvas stamp entrance
- Gold embossed tracked seal
- Session-level Legend recognition after the tracked effect settles

The face-following mark must not remain directly on the face for the entire stream. It should eventually detach and move into a safe persistent rail.

When no stable face is available:

- Do not stall the wheel.
- Use the last stable anchor briefly if still fresh.
- Otherwise use the configured upper-camera fallback anchor.
- Transition back to live tracking smoothly if the face returns during the active tracked phase.

### **Color Flood**

Implement one branded translucent fluid/tint treatment.

- Starts at a configured impact point
- Spreads farther based on tier
- Respects gameplay-safe masks
- Uses capped opacity
- Incorporates the username
- Leaves the appropriate signature afterward

### **Weather Control**

Implement one polished default weather treatment, such as falling stars or rain.

- Spin: local camera weather
- Power: complete camera weather
- Takeover: readable full-canvas weather
- Legend: premium weather entrance and session recognition

Use capped/reusable particle pools.

### **Break the Glass**

Implement one coherent glass-crack treatment.

- Local crack for smaller tiers
- Larger regional fracture for Power

- Brief username-forming full-canvas fracture for Takeover/Legend
- Etched persistent signature afterward

This is an overlay illusion; do not attempt to modify the underlying TikTok pixels.

## Steal the Spotlight

- Dim the overlay within a safe opacity range
- Search the canvas with a spotlight
- Land one light on the tracked face/camera anchor
- Reveal the gifter's name with another light
- Move the name into persistent recognition

Use tracking locally for the face spotlight.

## Claim the Frame

- Spin claims a small border treatment
- Power claims a larger border region
- Takeover receives a premium marquee
- Legend receives full-frame entrance plus Legend-rail recognition

Implement one coherent visual theme first.

## Avatar Invasion

- Use the sender avatar when available
- Implement one entrance, such as crashing through or peeking around the border
- Tier controls size, travel and persistence
- Provide a local fallback using the first grapheme of the username
- Avatar failure must never block queue progress

Sanitize/proxy/cache avatar data using the repository's existing safe pattern.

## Required gesture-controller face tracking

Face tracking must be built into the existing gesture controller.

This is not an optional future module.

## Camera/frame ownership

- The existing gesture controller must continue owning or consuming its current video feed.
- Do not call `getUserMedia()` a second time for face tracking.
- Do not introduce another camera-capture process.
- Do not replace TikTok LIVE Studio's camera source.

- Reuse each already-acquired frame for gesture and face inference.
- Avoid duplicate decoding or pixel copies where the existing architecture permits shared `VideoFrame`, `ImageBitmap`, canvas or texture use.

If the gesture controller already uses a combined face/hand/pose model, enable or reuse its face-landmark output rather than loading a redundant model.

If it currently recognizes only hands/gestures, add a face-landmark stage to the same acquisition/scheduling pipeline.

## Scheduling and performance

- Track one face only.
- Run face inference approximately 15–30 times per second.
- Let the overlay interpolate/smooth at render FPS.
- Analyze a reduced frame appropriate to the current controller.
- Preserve gesture responsiveness as the higher priority.
- Do not run expensive inference synchronously on the overlay animation thread.
- Reuse the controller's existing worker/off-main-thread design.
- If no worker exists and inference causes frame drops, introduce the smallest clean worker architecture that shares the existing frame source.

## Tracking output

Expose a normalized structure equivalent to:

```
interface FaceTrackingState {
  visible: boolean;
  confidence: number;
  timestamp: number;
  frameWidth: number;
  frameHeight: number;
  mirrored: boolean;
  bounds: {
    x: number;
    y: number;
    width: number;
    height: number;
  };
  anchors: {
    forehead: { x: number; y: number };
    headCenter: { x: number; y: number };
    nose: { x: number; y: number };
  };
  rotation: {
    roll: number;
  };
}
```

```
    yaw?: number;
    pitch?: number;
  };
  scale: number;
}
```

Use normalized coordinates unless the existing controller has a stronger established convention.

## Coordinate conversion

Correctly map tracker coordinates into the overlay's `1080×1920` coordinate system.

Account for:

- Camera source rectangle
- Input video dimensions
- Crop
- `cover` versus `contain`
- Letterboxing
- Mirroring
- Camera rotation
- Tracker orientation
- Overlay scaling
- Forehead offsets
- User calibration

Do not assume the raw landmark coordinate directly equals the visible TikTok camera coordinate.

Create a single tested transformation utility used by all tracked effects.

## Smoothing and loss handling

Reuse existing gesture-controller smoothing where appropriate or add a suitable EMA/One-Euro-style filter.

Provide configurable:

- Position smoothing
- Scale smoothing
- Rotation smoothing
- Confidence threshold
- Lost-face grace period
- Anchor offset
- Mirror-X
- Maximum jump distance
- Reacquisition blending

The stamp must feel attached rather than jittery, but must not visibly lag far behind normal head movement.

## Local transport only

Tracking data must travel through the gesture controller's existing local state/IPC mechanism.

If the gesture controller and overlay are in the same process, share the state directly.

If they are separate local clients, reuse or add an appropriate local mechanism such as:

- Existing localhost WebSocket
- Existing IPC bridge
- `BroadcastChannel`
- Shared worker/state service

Do not use Ably for tracking data.

If the production overlay is served over HTTPS, do not create a broken mixed-content dependency on an insecure `ws://localhost` connection. Reuse a compatible local deployment/IPC approach.

## Tracking diagnostics

The control interface must show:

- Gesture-controller connected/disconnected
- Face visible/lost
- Confidence
- Tracking update rate
- Tracking latency
- Current transformed forehead anchor
- Mirror/crop mode
- Optional debug landmark/bounds overlay
- Calibration controls

Debug landmarks must never appear in production mode.

## Persistent signature registry

Separate earned persistence from visible screen slots.

Every active signature must remain registered until its promised expiry even if it is temporarily rotated out of view.

Requirements:

- Exclusive arrival visibility for approximately 3–5 seconds
- Regular-signature carousel
- Premium-signature carousel
- Legend rail supporting multiple active Legends
- Hidden-by-rotation does not equal expired
- No active signature is evicted before its promised expiry
- Latest or strongest Legend may occupy the primary position while other Legends remain in rotation
- Explicit session reset is required to remove session-level signatures

Persist:

- Event ID
- Session ID
- User identity
- Result
- Tier
- Variant
- Style seed
- Created time
- Expiry time
- Signature group
- Display priority
- Avatar reference where applicable

The backend/controller is authoritative. Browser `localStorage` may be a cache, but not the sole source of truth.

## State recovery

On overlay startup/reconnect:

1. Subscribe or begin buffering new revisions.
2. Hydrate the authoritative session snapshot.
3. Apply buffered events newer than the snapshot revision.
4. Restore active signatures and pending queue items.
5. Do not replay already-completed impact animations.
6. Resume pending spins safely.

Use monotonic revisions or an equivalent existing mechanism to prevent stale state from overwriting newer state.

## Geometry and safe areas

Use one internal `1080×1920` coordinate system.

Create configurable geometry for:

- Camera rectangle
- Gameplay rectangle
- Camera/game boundary
- Left and right signature rails
- Top and bottom safe regions
- TikTok right-side UI exclusion zone
- TikTok bottom UI exclusion zone
- Wheel idle position
- Wheel active position

Provide numeric calibration and an on-screen calibration mode.

Scale the complete overlay proportionally when rendered at a different preview size.

## Operator interface

Required controls:

- Armed/disarmed
- Live/simulation mode
- Start session
- End session
- Confirmed session reset
- Current event
- Queue count
- Pause/resume intake
- Skip current
- Drain queue
- Clear queue
- Emergency clear transient effects
- Clear persistent signatures separately
- Replay last event
- Trigger any result/tier manually
- Enable/disable individual results
- Configure tier thresholds
- Configure durations and maximum opacity
- Rapid-spin threshold
- Low-intensity/reduced-motion mode
- Sound mute/volume if the existing audio route supports it
- Active-signature management
- Geometry calibration
- Face-tracking calibration and diagnostics
- Ably connection state
- Last valid Gift-event time



- Renderer FPS and active particle count

Use the existing control interface and design system. Do not create an unrelated second dashboard.

The test interface must use the same production renderer and lifecycle code, not a separate visual imitation.

## Idle discoverability

Provide a configurable compact idle treatment so a new viewer can understand that the wheel exists without occupying significant space.

Suggested copy:

GIFTS SPIN THE SCREEN

It may include a small static or gently moving wheel icon. Avoid constant distracting motion.

When a Gift is accepted, the compact treatment expands into the active wheel.

## Visual and accessibility requirements

- Transparent 1080×1920 background
- `pointer-events: none` on the production overlay
- Mobile-readable outlined text
- No external font dependency required during a LIVE
- Blue, orange, purple, white and gold default palette
- No red/green-only status communication
- No rapid flashing
- No sustained gameplay blur
- Full-screen opaque obstruction kept extremely brief
- Configurable low-intensity mode
- Particle caps and reusable pools
- No unbounded DOM creation
- No reliance on CSS `backdrop-filter` to affect separate TikTok sources
- Safe cleanup of timers, animations, images and tracking subscriptions
- Feature detection and graceful fallback for unsupported browser APIs

Do not add audio until the existing TikTok/OBS/Wave Link routing has been inspected. Avoid creating an effect sound that reaches only one platform or is duplicated.

## Acceptance tests

Implement automated tests appropriate to the existing stack plus a visual test matrix.

The completed work must demonstrate:

1. One hundred duplicate deliveries of one event produce exactly one spin.
2. One hundred preselected wheel tests land on the correct slice.
3. A twenty-event burst causes no silent loss and remains operable.
4. Every queued sender receives an immediate acknowledgment.
5. Completed Gift streaks produce one correctly valued spin.
6. Disabled results are neither selected nor displayed.
7. All 32 result/tier combinations are available in the test matrix.
8. Refresh restores active signatures without replaying completed impacts.
9. Refresh restores pending queue items safely.
10. Long, Unicode, emoji and right-to-left usernames render safely.
11. Missing or failed avatars use the fallback.
12. A failed animation cleans up and releases the queue within one second.
13. Emergency clear removes transient effects within 500 ms.
14. Emergency clear does not erase persistent signatures.
15. No per-frame tracking or animation data is sent through Ably.
16. Face tracking uses the gesture controller's existing frame acquisition.
17. Face tracking does not open a second camera.
18. Tracking continues locally during temporary Ably/network loss.
19. Mirrored, cropped and `cover` camera layouts map the forehead anchor correctly.
20. Temporary face loss uses a stable fallback without blocking the wheel.
21. Reacquisition blends smoothly without a visible jump.
22. No strong full-screen obstruction exceeds the configured maximum.
23. The overlay remains transparent outside active artwork.
24. A 30-minute synthetic soak shows no monotonic growth in DOM nodes, timers, particles, subscriptions or memory.
25. `1080×1920` and proportionally scaled previews preserve geometry.
26. The original TikTok LIVE Studio → projector → OBS route remains unchanged.
27. Formatter, linter, type checker, tests and production build all pass.

## Required deliverables

Complete and provide:

- Working transparent wheel overlay
- Authoritative tier and result resolver
- Gift streak normalization
- Queue and deduplication
- Separate receipt, wheel, impact, ambient and signature lanes
- All eight launch effects with one polished variant each
- Fully polished Stamp Your Name effect
- Face tracking integrated into the existing gesture controller
- Tested coordinate transformation and calibration
- Persistent signature registry
- Refresh/reconnect recovery

- Operator controls
- Manual event simulator and 32-combination visual matrix
- Automated tests
- Production build
- Setup documentation
- Exact TikTok LIVE Studio overlay setup steps
- Gesture-controller tracking configuration steps
- Pre-LIVE testing checklist
- Concise list of changed files and architectural decisions

Do not leave placeholder effects or a face-tracking interface without an implementation. Do not stop after the repository audit unless the gesture-controller source is genuinely missing.